DAY 3

Filtering Rows with WHERE

30-Day PostgreSQL for Data Engineering

Session	Day 3 — Filtering Rows with WHERE
Duration	~45 minutes
Level	Beginner — no prior SQL needed
Week	Week 1 — SQL Foundations
Goal	Write WHERE clauses to filter rows precisely
Database	PostgreSQL 16 (Neon cloud)

■ What You Will Learn Today

- Use the WHERE clause to filter rows from a table
- Apply all six comparison operators: =, !=, >, <, >=, <=
- Handle NULL values correctly with IS NULL and IS NOT NULL
- Understand the pipeline use case for each filtering pattern
- Combine WHERE with SELECT and AS (Day 2 skills) in real queries

Session Agenda

#	Segment	Time	Description
1	Intro & recap	0:00–3:00	Welcome back, Day 2 recap, today's goals
2	The WHERE clause	3:00–10:00	Syntax, position, how filtering works
3	Comparison operators	10:00–20:00	=, !=, >, <, >=, <= with live demos

4	NULL values	20:00–30:00	IS NULL / IS NOT NULL — the tricky one
5	Data type gotchas	30:00–36:00	Quotes for strings, no quotes for numbers
6	Hands-on exercise	36:00–43:00	Students write their own filters
7	Recap & Day 4 preview	43:00–45:00	Summary + what's coming tomorrow

Segment 1 — Intro & Recap [0:00–3:00]

Camera script:
"Welcome back to Day 3! Yesterday you learned how to SELECT columns and alias them with AS. Today we're going to make our queries actually useful — by filtering rows. In the real world, you almost never want ALL the data. You want *specific* rows. That's exactly what WHERE does. Let's get into it."

Quick Day 2 recap (30 seconds):

- SELECT picks columns, FROM picks the table
- SELECT * returns every column
- AS renames columns in the output (aliases)

Segment 2 — The WHERE Clause [3:00–10:00]

Talking points:

- WHERE goes AFTER FROM — always. SELECT → FROM → WHERE is the order.
- WHERE acts like a filter or gatekeeper: only rows passing the condition flow through.
- If no rows match, you get an empty result — not an error.
- The condition evaluates to TRUE or FALSE for each row. TRUE = include, FALSE = exclude.

Basic syntax:

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Live demo — show on screen:

```
-- First: no filter (all rows)
SELECT order_id, status, amount FROM orders;

-- Then: add WHERE
SELECT order_id, status, amount
FROM orders
WHERE status = 'shipped';
```

Pipeline connection: "In incremental pipelines, WHERE is how you grab only new data — for example WHERE loaded_at > last_run_timestamp. This is one of the most important SQL patterns in data engineering."

Segment 3 — Comparison Operators [10:00–20:00]

Comparison operators reference:

Operator	Meaning	Example
`=`	Equal to	`status = 'shipped'`
`!=` or `<>`	Not equal to	`status != 'cancelled'`
`>`	Greater than	`amount > 100`
`<`	Less than	`amount < 50`
`>=`	Greater than or equal	`order_date >= '2024-01-01'`
`<=`	Less than or equal	`price <= 9.99`

Demo queries — run each one live:

```
-- Greater than (numeric)
SELECT order_id, amount FROM orders WHERE amount > 200;

-- Less than or equal (numeric)
SELECT order_id, amount FROM orders WHERE amount <= 100;

-- Not equal (text)
SELECT order_id, status FROM orders WHERE status != 'cancelled';

-- Date comparison (incremental load pattern)
SELECT order_id, order_date FROM orders WHERE order_date >= '2024-06-01';

-- Exact string match
SELECT order_id, country FROM orders WHERE country = 'Canada';
```

Talking points:

- Text (strings) always need SINGLE quotes: 'Canada' not "Canada"
- Numbers never need quotes: amount > 200 not amount > '200'
- Dates use single quotes too: '2024-06-01'
- Both != and <> work in PostgreSQL — != is more common in practice

Segment 4 — NULL Values [20:00–30:00]

NULL is one of the trickiest things in SQL. Spend time here — students often get burned by it.

What is NULL?

- NULL means UNKNOWN or MISSING — not zero, not empty string, not false
- A NULL amount is not $0. A NULL email is not an empty email.
- NULL is the absence of any value whatsoever

The critical rule — show this on screen:

```
-- ■ This NEVER works — returns no rows even if NULLs exist!
SELECT * FROM orders WHERE shipped_at = NULL;

-- ■ Correct: find rows where shipped_at is missing
SELECT * FROM orders WHERE shipped_at IS NULL;

-- ■ Correct: find rows where shipped_at exists
SELECT * FROM orders WHERE shipped_at IS NOT NULL;
```

Camera script:
"NULL is contagious — any comparison with NULL returns NULL, not TRUE or FALSE. That's why = NULL never works. PostgreSQL gives you IS NULL and IS NOT NULL specifically for this. Memorize these — you'll use them constantly."

Pipeline use case — highlight this:

```
-- Classic pipeline queue pattern:
-- Find all records that haven't been processed yet
SELECT row_id, created_at
FROM staging_events
WHERE processed_at IS NULL;
-- These are your "work to do" rows
```

Segment 5 — Data Type Gotchas [30:00–36:00]

Quick reference — quotes by type:

Data Type	Example Filter	Quotes?
TEXT / VARCHAR	`WHERE country = 'Canada'`	Yes — single quotes
INTEGER / NUMERIC	`WHERE amount > 100`	No quotes
DATE / TIMESTAMP	`WHERE order_date >= '2024-01-01'`	Yes — single quotes
BOOLEAN	`WHERE is_active = TRUE`	No quotes

Case sensitivity note:

```
-- PostgreSQL string comparisons ARE case-sensitive by default!
WHERE status = 'Shipped' -- won't match 'shipped'
WHERE status = 'shipped' -- matches 'shipped'

-- Tip: Day 6 covers LOWER() to handle this safely
```

Segment 6 — Hands-On Exercise [36:00–43:00]

Setup — paste this first:

```
CREATE TABLE orders (
order_id INTEGER,
customer_id INTEGER,
status TEXT,
amount NUMERIC(10,2),
country TEXT,
order_date DATE,
shipped_at TIMESTAMP
);

INSERT INTO orders VALUES
(1, 101, 'shipped', 250.00, 'Canada', '2024-05-10', '2024-05-12 09:00:00'),
(2, 102, 'pending', 45.50, 'USA', '2024-06-01', NULL),
(3, 103, 'shipped', 310.00, 'UK', '2024-06-03', '2024-06-05 14:30:00'),
(4, 104, 'cancelled', 99.99, 'Canada', '2024-04-20', NULL),
(5, 105, 'pending', 180.00, 'USA', '2024-06-15', NULL),
(6, 106, 'shipped', 520.75, 'Germany', '2024-06-20', '2024-06-22 11:00:00');
```

Exercise tasks — say these out loud:

Task	Description	Expected
1 — Basic string filter	Find all orders where status = 'shipped'	3 rows
2 — Numeric comparison	Find orders where amount > 200	3 rows
3 — IS NULL	Find orders that have not been shipped yet (shipped_at IS NULL)	3 rows
4 — Date filter	Find orders placed on or after '2024-06-01'	4 rows
5 — Challenge	Find orders from Canada with amount > 100	2 rows

Expected output — Exercise 1:

```
-- WHERE status = 'shipped'
order_id | customer_id | status | amount | country | order_date | shipped_at
----------+-------------+---------+--------+---------+------------+-------------
--------
1 | 101 | shipped | 250.00 | Canada | 2024-05-10 | 2024-05-12 09:00:00
3 | 103 | shipped | 310.00 | UK | 2024-06-03 | 2024-06-05 14:30:00
```

```
6 | 106 | shipped | 520.75 | Germany | 2024-06-20 | 2024-06-22 11:00:00
```

Common errors to watch for:

Common Error	Why it happens	Fix
WHERE status = NULL	NULL cannot be compared with =	Use IS NULL instead
WHERE amount = "100"	Double quotes are for identifiers	Numbers need no quotes: amount = 100
WHERE name = 'O'Brien'	Single quote breaks the string	Escape with two quotes: 'O''Brien'
No rows returned	Value typed wrong or wrong case	Check exact value: 'Shipped' vs 'shipped'

Segment 7 — Recap & Day 4 Preview [43:00–45:00]

Recap — key points:

- WHERE filters rows — only rows matching the condition are returned
- Comparison operators: =, !=, >, <, >=, <=
- Strings need single quotes. Numbers and booleans do not.
- NULL means missing/unknown — use IS NULL and IS NOT NULL, never = NULL
- WHERE is the backbone of pipeline queries — incremental loads, queues, filters

Camera script — Day 4 preview:

"Tomorrow on Day 4 we're going to supercharge your WHERE clauses with logical operators. Right now you can only filter on ONE condition at a time. Tomorrow you'll combine them — AND, OR, NOT, IN, and BETWEEN. That means queries like: find orders from Canada OR the UK placed after June 1st. See you in Day 4!"

Appendix — Quick Reference Card

Syntax	Example	Notes
`WHERE col = value`	`WHERE status = 'active'`	Strings: single quotes
`WHERE col != value`	`WHERE status != 'failed'`	Also: <>
`WHERE col > number`	`WHERE amount > 100`	No quotes for numbers
`WHERE col >= date`	`WHERE dt >= '2024-01-01'`	Dates: single quotes
`WHERE col IS NULL`	`WHERE shipped_at IS NULL`	NOT: col = NULL
`WHERE col IS NOT NULL`	`WHERE email IS NOT NULL`	Rows with actual values

Glossary

Term	Definition
`WHERE`	Clause that filters rows based on a condition — only matching rows are returned
`Comparison operator`	Symbol that compares two values: =, !=, >, <, >=, <=
`NULL`	The absence of a value — not zero, not empty string, just unknown/missing
`IS NULL`	Tests whether a value is missing (NULL). Never use = NULL.
`IS NOT NULL`	Tests whether a value exists (is not missing)
`Predicate`	The condition expression after WHERE that evaluates to TRUE or FALSE
`Incremental load`	Pipeline pattern: WHERE loaded_at > last_run loads only new rows